Manual de Arquitectura de Software y Documentación Técnica

Sistema de Optimización de Preparación de Pedidos por Olas (*Wave Picking*) y Simulación Multi-Picker por Zonas

---

1. Introducción y Propósito del Sistema

El sistema implementado es una solución integral de ingeniería de software para centros de distribución y almacenes logísticos inteligentes (*Warehouse Management Systems* - WMS). Su objetivo primordial es **optimizar el proceso de preparación de pedidos (*picking*)** mediante la técnica de **Wave Picking (Agrupamiento por Olas)**, asistida por una **zonificación espacial del almacén**, **enrutamiento inteligente sobre grafos dirigidos (2D/3D)** y un **motor de simulación multi-agente con evasión reactiva de tráfico**.

El software permite:

1. **Modelar y parametrizar un almacén físico**: Racks de estiba, pasillos de circulación de mano única o doble, puertas de carga/descarga y zonas de despacho con límites de carga.
2. **Validar formalmente restricciones geométricas y topológicas**: Asegurar que todos los productos sean alcanzables y que la red de pasillos sea fuertemente conexa (sin callejones sin salida ni trampas de circulación).
3. **Agrupar pedidos en Waves (*Batching & Order Splitting*)**: Respetar capacidades de peso, volúmenes de pedidos y gestionar quiebres de stock con reubicación inteligente de productos.
4. **Dividir el trabajo en sublistas por Picker según su Zona de trabajo**: Garantizar especialización zonal y balanceo de carga.
5. **Calcular rutas óptimas de recolección**: Comparar tres estrategias de pathfinding (**A**: Orden de Pedidos, **B**: Cercanía Greedy 3D, **C**: Barrido por Pasillos en S-Shape) considerando penalizaciones por altura de estante (*Z*).
6. **Coordinar una flota de Pickers en paralelo**: Simulación continua con detección de colisiones, maniobras de desvío dinámico, descarga en la zona más cercana y asistencia cooperativa inter-zona.
7. **Presentar un Dashboard visual interactivo**: Interfaz gráfica en tiempo real con zoom responsivo, control de reproducción paso a paso y comparador de métricas.

---

2. Organización de Archivos y Módulos

El proyecto se estructuró bajo el principio de **Responsabilidad Única (SRP)** y **Bajo Acoplamiento**, organizando el código en módulos especializados dentro de `wave_picking_src/`:

```
PROYECTO-WAVE/
├── data/                          # Escenarios serializados en JSON
│   ├── escenario_basico.json      # Almacén base con configuración estándar
│   └── escenario_grande.json      # Almacén de alta escala
└── wave_picking_src/              # Código fuente modular
    ├── main.py                    # Punto de entrada y orquestador de inicio
    ├── estructuras.py             # Estructuras de datos nativas (Tabla Hash, Heap, MergeSort)
    ├── modelos.py                 # Entidades del dominio (Warehouse, Picking, Waves)
    ├── warehouse.py               # Topología celular, Grafo dirigido y Validación formal
    ├── algoritmos.py              # Lógica de Wave Picking, Zonificación y Order Splitting
    ├── rutas.py                   # Dijkstra, Pathfinding 3D y Estrategias A, B y C
    ├── coordinador.py             # Motor de eventos discretos, tráfico y multi-picker
    ├── persistencia.py            # Adaptador JSON (Guardar/Cargar escenarios y estados)
    ├── generador.py               # Factoría procedural de almacenes matemáticamente válidos
    └── simulacion.py              # Interfaz Gráfica de Usuario (Tkinter/ttk GUI)
```

---

3. Arquitectura por Capas y Jerarquía de Dependencias

Para lograr una clara **separación de intereses (*Separation of Concerns*)**, el sistema adopta una arquitectura en **4 Capas Jerárquicas Estrictas**. Una capa superior puede consumir servicios de las capas inferiores, pero **ninguna capa inferior conoce la existencia de las capas superiores**:



Relación y orden de las capas (de menor a mayor nivel de abstracción):

1. **Capa 1: Infraestructura y Estructuras de Datos Base (`estructuras.py`)**
   - **Nivel más bajo del sistema**. No depende de ninguna otra clase del proyecto.
   - Provee las estructuras algorítmicas fundamentales:
     - `TablaHash`: Implementación propia de tabla hash con resolución de colisiones por encadenamiento (`NodoHash`) y redimensionamiento dinámico automático (duplicación de capacidad al superar el factor de carga de 0.75).

- `ColaPrioridad`: Min-Heap binario con métodos de flotar (`_flotar`) y hundir (`_hundir`) para optimizar Dijkstra en $O(ElogV)$.
- `merge_sort`: Algoritmo de ordenamiento determinista Divide y Vencerás con complejidad temporal $O(NlogN)$.

2. **Capa 2: Dominio, Entorno Topológico y Algoritmos Centrales (`modelos.py`, `warehouse.py`, `rutas.py`, `algoritmos.py`)**
   - **`modelos.py`**: Modela los objetos de negocio puros (`Rack`, `Pasillo`, `Puerta`, `ZonaDespacho`, `Picker`, `Producto`, `Pedido`, `ItemPedido`, `Wave`, `LineaRecoleccion`).
   - **`warehouse.py`**: Construye la grilla bidimensional discretizada y el `GrafoWarehouse` dirigido (usando `TablaHash` para las listas de adyacencia). Implementa la validación formal de las 6 reglas operativas del almacén y la garantía de conectividad fuerte (eliminación de trampas de flujo).
   - **`rutas.py`**: Resuelve el camino mínimo con `dijkstra`, soporte de caché y bloqueo de nodos. Implementa las estrategias de ruteo **A** (Secuencial), **B** (Greedy 3D) y **C** (S-Shape por pasillos) con penalización por altura vertical (Z).
   - **`algoritmos.py`**: El `WaveManager` ejecuta la partición del espacio en zonas equitativas para cada picker, gestiona quiebres de stock buscando ubicaciones alternativas y agrupa pedidos en olas fraccionadas (*Order Splitting*).

3. **Capa 3: Orquestación, Simulación Continua y Servicios (`coordinador.py`, `generador.py`, `persistencia.py`)**
   - **`coordinador.py`**: El `CoordinadorMultiPicker` actúa como motor de eventos discretos. Gestiona el avance paso a paso de múltiples pickers simultáneos, previene colisiones frontales y cruces mediante desvíos en tiempo real con Dijkstra, descuenta el peso en las zonas de despacho y coordina la cooperación entre zonas.
   - **`persistencia.py`**: Implementa el patrón Adaptador para serializar y deserializar el estado del almacén y las simulaciones en archivos JSON.
   - **`generador.py`**: Factoría procedural que calcula dimensiones, estantes, calles y puertas, garantizando que el escenario generado cumpla todas las restricciones estructurales.

4. **Capa 4: Presentación e Interfaz de Usuario (`simulacion.py`, `main.py`)**
   - **Capa superior**. Contiene `AppVisual`, una interfaz construida en Tkinter con diseño responsivo.
   - **Desacople total de la lógica**: La GUI **nunca** calcula caminos mínimos ni decide a qué estante debe ir un picker. Únicamente invoca a la Capa 3 (`CoordinadorMultiPicker`, `WaveManager`, `PersistenciaJSON`), recibe matrices de estados (`planes_activos`, `cap_zonas_por_paso`) y las renderiza en el canvas vectorial y las tablas informativas.

---

4. Separación entre la Lógica de Negocio y la Interfaz Gráfica

Uno de los logros arquitectónicos más importantes del sistema es la **independencia absoluta de la lógica respecto al entorno visual**:

Aspecto	**Capa de Negocio / Coordinación**	**Capa Gráfica (`AppVisual`)**
Rol	Cálculo matemático, ruteo, división de waves y resolución de choques.	Captura de eventos del usuario, renderizado gráfico y animación.

Aspecto	Capa de Negocio / Coordinación	Capa Gráfica (`AppVisual`)
Dependencia de Tkinter	**Cero**. Ni `warehouse.py`, ni `algoritmos.py`, ni `coordinador.py` importan librerías gráficas.	Importa `tkinter` y `ttk`.
Entrada y Salida	Recibe objetos del modelo (`Warehouse`, `Wave`) y retorna estructuras de datos puras (diccionarios, listas de tuplas de coordenadas, métricas).	Recibe las estructuras del coordinador e invoca métodos de dibujo (`canvas.create_rectangle`, `canvas.create_oval`).
Ejecución Headless	Puede correrse en scripts automáticos, pruebas unitarias o servidores sin entorno gráfico.	Es un cliente consumidor reemplazable (podría cambiarse por una interfaz web o Qt sin tocar la lógica de negocio).

El Patrón Snapshot (Planes Precalculados y Paso a Paso)

En lugar de acoplar el bucle gráfico con el algoritmo de Dijkstra en cada milisegundo:

1. Cuando el usuario presiona "Iniciar Simulación" o selecciona una estrategia, la GUI solicita al `CoordinadorMultiPicker.planificar_simulacion_continua(...)` que compute toda la simulación.

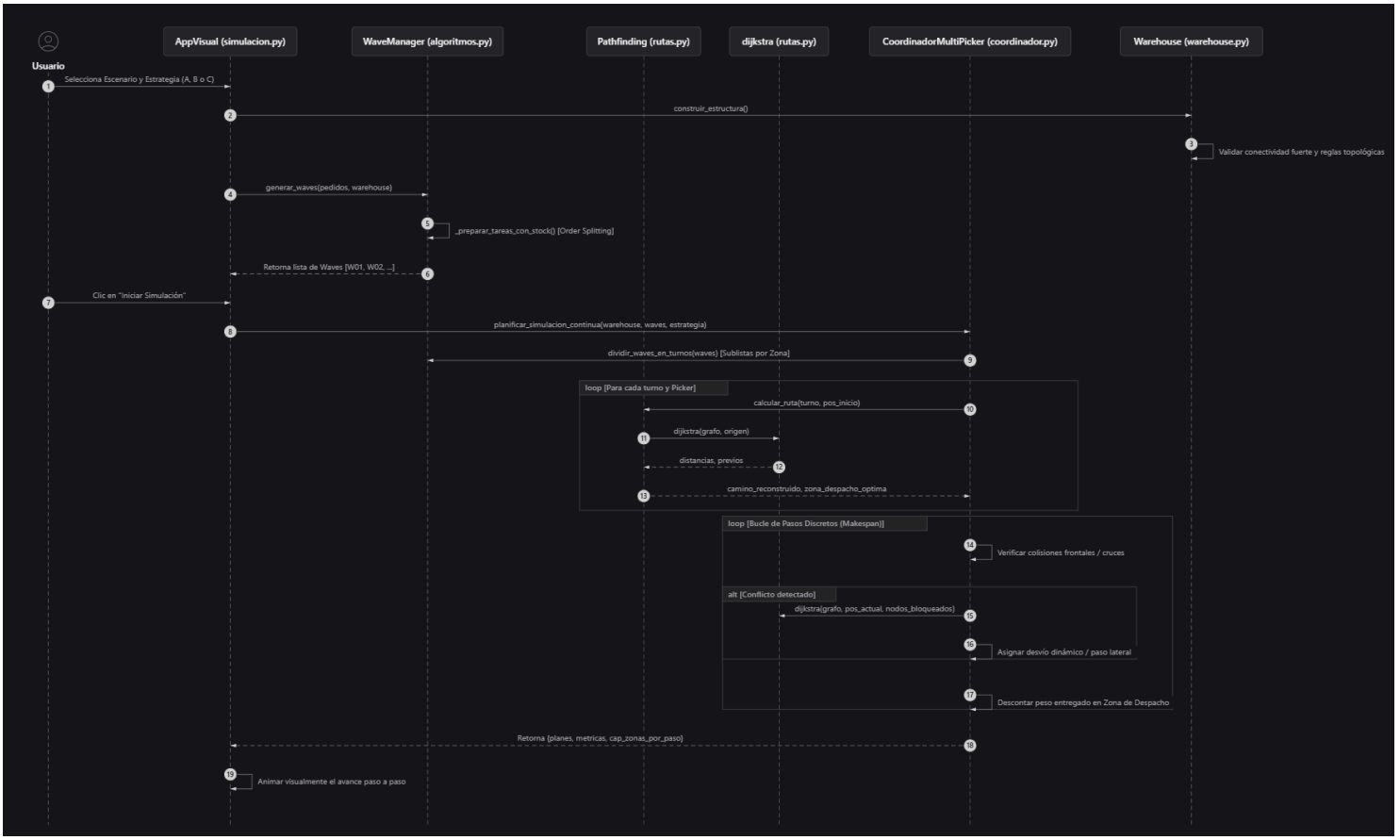
2. El coordinador retorna un objeto integral de simulación:
   - `planes`: Contiene para cada picker el historial de coordenadas `(x, y)` por cada paso de tiempo, eventos ocurridos (recolección, desvíos, entregas), peso cargado y productos activos.
   - `cap_zonas_por_paso`: Registro del inventario de peso disponible en cada Zona de Despacho en cada instante.
   - `metricas`: Tiempos de cómputo, operaciones elementales, distancia total recorrida y desvíos evitados.

3. La interfaz gráfica se limita a iterar sobre ese historial mediante un temporizador (`self.after(ms, self._animar_paso)`), actualizando las posiciones de los avatares y los colores de las celdas.

---

5. Interacción entre las Clases Principales

A continuación se detalla cómo colaboran las clases centrales a lo largo del ciclo de vida del software:



---

6. Explicación Detallada de los Subsistemas de Lógica

6.1. Zonificación del Almacén por Picker (`WaveManager.obtener_zonas_pickers`)

- El eje *X* del almacén se divide equitativamente entre la cantidad de pickers disponibles.
- Por ejemplo, en un almacén de 18 columnas de ancho con 2 pickers:
  - **Picker 1**: Sector izquierdo ($X \in [0, 8]$).
  - **Picker 2**: Sector derecho ($X \in [9, 17]$).
- A cada producto se le asigna su zona correspondiente según la coordenada *X* de su rack. Cada picker atiende con prioridad exclusiva las tareas de su propia zona.

6.2. Agrupamiento de Pedidos y *Order Splitting* (`WaveManager.generar_waves`)

1. **Control de Prioridades**: Los pedidos pendientes se ordenan mediante `merge_sort` en forma decreciente por prioridad ($O(NlogN)$).

2. **Quiebre de Stock y Reemplazo**: Si un producto no tiene stock suficiente en su ubicación original, el algoritmo busca ubicaciones alternativas con el mismo producto dentro del almacén minimizando la distancia Manhattan.

3. **Restricciones Multidimensionales de la Wave**:

- Cantidad máxima de pedidos por ola ($\leq wave_max_pedidos$).
- Cantidad máxima de unidades ($\leq wave_max_productos$).
- Límite de peso total admitido por las Zonas de Despacho ($\leq peso_max_kg$).

4. **Fraccionamiento (*Order Splitting*)**: Si un pedido no cabe completamente en una ola debido al límite de peso o de unidades, se fracciona atómicamente: las unidades que entran se consolidan en la ola actual y el remanente continúa pendiente para la siguiente ola.

6.3. Algoritmos de Ruteo y Estrategias de Recorrido (`rutas.py`)

El almacén se modela como un **Grafo Dirigido** $G = (V, E)$, donde cada celda de pasillo es un nodo y las aristas respetan estrictamente el sentido de circulación asignado (Norte ↑, Sur ↓, Este $\rightarrow$, Oeste $\leftarrow$ o Bidireccional).

El sistema implementa tres estrategias para visitar los productos de una Wave:

- **Estrategia A (Secuencial por Pedido)**: Visita las ubicaciones exactamente en el orden cronológico en que fueron listadas por el cliente.
- **Estrategia B (Cercanía Greedy 3D)**: En cada punto, consulta Dijkstra y selecciona la siguiente ubicación no visitada que minimice el costo 3D:

$$Costo_{3D} = Distancia\ en\ Planta\ (Grafo) + max(0, Z_{estante} - 1)$$

- **Estrategia C (Barrido por Pasillos / S-Shape Bilateral)**: Agrupa los productos por el pasillo que los asiste y los ordena en el sentido de avance del pasillo. Permite recolección bilateral (acceder a estantes a izquierda y derecha en una sola pasada de la calle).

Al completar la lista de recolección, el picker se dirige automáticamente a la **Zona de Despacho más cercana que cuente con capacidad de peso remanente**.

6.4. Coordinación Multi-Picker y Evasión Dinámica de Tráfico (`coordinador.py`)

En un entorno con varios recolectores operando simultáneamente, pueden producirse conflictos espaciales:

- **Colisión Frontal / Mismo Destino**: Dos pickers intentan ingresar a la misma celda en el mismo paso.
- **Cruce Invertido**: Dos pickers en celdas contiguas intentan intercambiar posiciones.

El `CoordinadorMultiPicker` resuelve esto mediante el siguiente protocolo de evasión:

1. Si la celda objetivo está libre, el picker avanza normalmente.
2. Si la celda objetivo está ocupada, se marca temporalmente dicha celda como **nodo bloqueado**.
3. Se invoca a `dijkstra(warehouse.grafo, pos_actual, nodos_bloqueados=celdas_ocupadas)` para calcular un camino alternativo hacia el siguiente objetivo de recolección.
4. Si existe un camino de circunvalación, el picker toma el desvío. Si no es posible desviar (por restricciones de flechas en el pasillo), realiza un movimiento de escape lateral o espera un turno hasta que la vía quede liberada.

6.5. Cooperación Inter-Zona (*Dynamic Helping*)

Cuando la opción de ayuda entre zonas está habilitada:

- Si el Picker 1 finaliza todas las líneas de recolección de su mitad del almacén mientras el Picker 2 aún tiene trabajo pendiente, el Picker 1 cruza la frontera de zona para auxiliarlo.
- El coordinador divide dinámicamente el turno restante del Picker 2 respetando la capacidad de carga libre del Picker 1 y recalcula las rutas de ambos agentes sin interrupciones.

---

7. Verificación Formal del Warehouse (Reglas Operativas)

El módulo `warehouse.py` contiene el método `validar_escenario()`, el cual audita matemáticamente que el almacén cumpla con las 6 restricciones requeridas para una operación segura y sin bloqueos:

1. **Límites Físicos y Cero Solapamiento**: Ningún rack, pasillo, zona o puerta puede exceder las dimensiones de la grilla (W_X, L_Y) ni superponerse espacialmente sobre otra entidad.

2. **Geometría de Racks con Lado 2**: Todo rack debe medir exactamente 2 unidades en al menos una de sus dos dimensiones (ancho o alto), garantizando acceso directo a los ítems desde un pasillo adyacente.

3. **Puertas Perimetrales**: Todas las puertas de acceso deben situarse obligatoriamente sobre el perímetro exterior del almacén ($X \in \{0, W_X - 1\}$ o $Y \in \{0, L_Y - 1\}$).

4. **Zonas de Despacho Frontales**: Delante de cada puerta debe existir una Zona de Despacho adyacente para maniobras de estiba y control de carga.

5. **Inventario Contenido en Racks**: Todo producto registrado debe encontrarse estrictamente dentro de las coordenadas de un rack válido.

6. **Conectividad Fuerte y Accesibilidad 100%**: Mediante dos recorridos en anchura (BFS hacia adelante y BFS inverso sobre el grafo transpuesto), se verifica que desde la zona de despacho sea posible llegar a todas las caras de acceso de los racks y retornar al despacho respetando el sentido de las flechas de circulación.

---

8. Resumen de Flujo de Datos



Conclusión

La arquitectura del sistema garantiza:

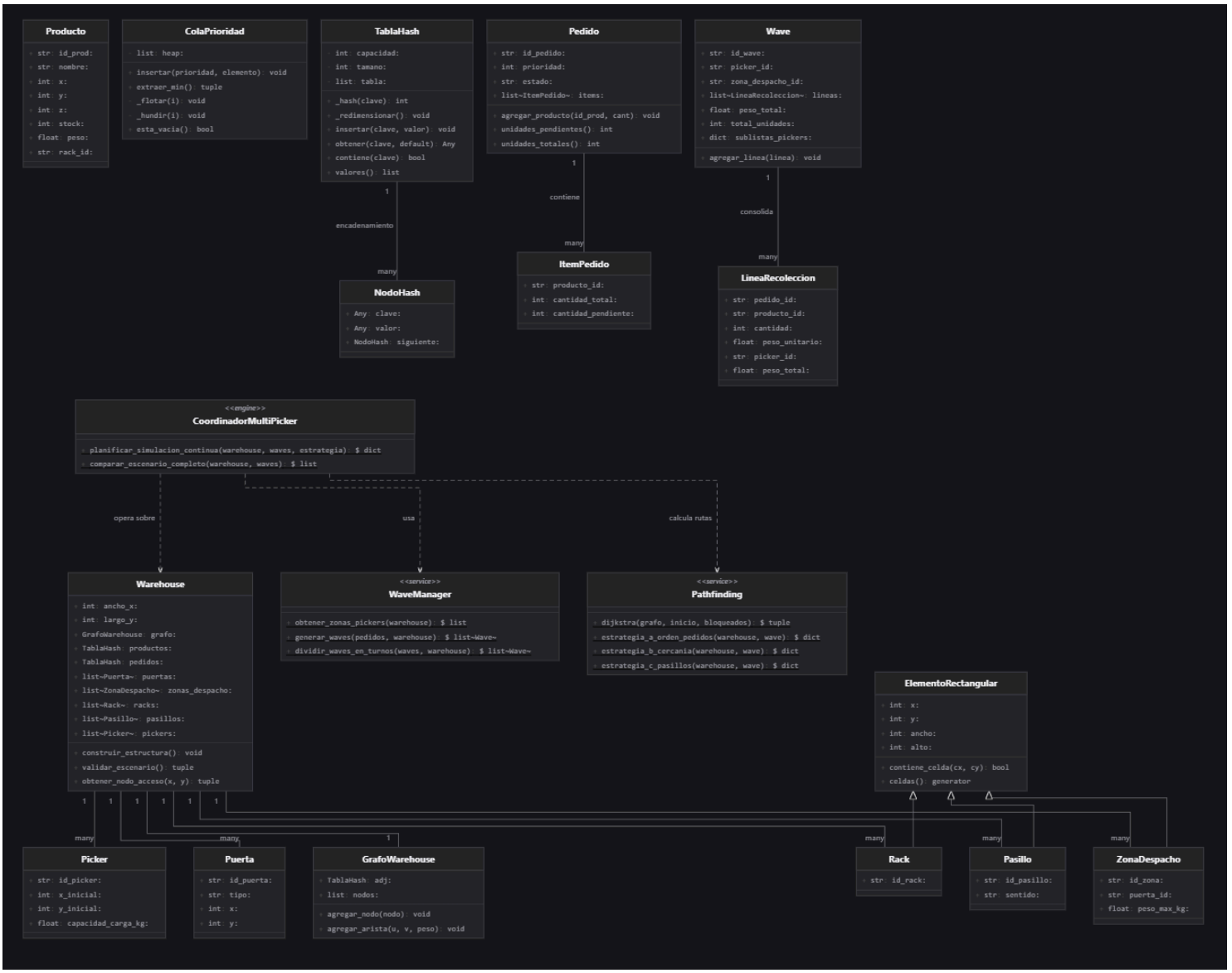
- **Mantenibilidad**: Código desacoplado en módulos de menos de 500 líneas con interfaces claras.
- **Rendimiento**: Estructuras nativas eficientes ($O(1)$ en búsquedas hash y $O(ElogV)$ en caminos mínimos con min-heaps).

- **Extensibilidad**: Capacidad de incorporar nuevas estrategias de ruteo, tipos de vehículos o algoritmos de batching sin modificar la interfaz gráfica ni las entidades fundamentales del dominio.

---

9. Diagrama de Clases UML y Modelo de Datos Detallado

Para comprender la estructura interna orientada a objetos del sistema, a continuación se presenta el diagrama de clases completo con las relaciones de herencia, composición y agregación entre los módulos centrales:



---

10. Análisis Matemático y de Complejidad Algorítmica

El diseño del software prescindió intencionalmente de bibliotecas de grafos externas (como `networkx`) o colecciones preconstruidas, implementando estructuras nativas optimizadas para el dominio:

10.1. Tabla Hash Auto-Redimensionable (`estructuras.py`)

- **Función de Dispersión para Coordenadas 2D**:
Para indexar nodos del grafo representados por tuplas (x, y), se implementó una función de dispersión espacial basada en números primos grandes:
$$h(x, y) = ((x \cdot 73856093) \oplus (y \cdot 19349663)) \ (mod\ M)$$
Esta función garantiza una distribución pseudoaleatoria uniforme sobre la grilla celular, minimizando la concentración de colisiones en pasillos contiguos.

- **Función de Dispersión Polinómica para Cadenas (IDs de productos y pedidos)**:
$$h(s) = \left(\sum_{i=0}^{k-1} s[i] \cdot 31^{k-1-i} \right) \pmod{M}$$

- **Factor de Carga (α) y Redimensionamiento**:
$$\alpha = \frac{N}{M}$$
El factor de carga se mantiene estrictamente en $\alpha \leq 0.75$. Al superarse, se ejecuta un rehasheo completo en una tabla de tamaño $M' = 2M + 1$.

  - **Búsqueda, Inserción y Borrado**: $O(1)$ en promedio.
  - **Peor caso**: $O(N)$ (degradación mitigada por redimensionamiento y números primos).

10.2. Cola de Prioridad Binaria - Min Heap (`estructuras.py`)

Utilizada como la estructura vehicular para el algoritmo de Dijkstra:

- **Almacenamiento**: Arreglo lineal contiguo donde para un nodo en el índice i:
  - Hijo izquierdo: $2i + 1$
  - Hijo derecho: $2i + 2$
  - Padre: $\lfloor (i - 1)/2 \rfloor$
- **Inserción (`_flotar`)**: Complejidad temporal $O(logV)$.
- **Extracción del Mínimo (`_hundir`)**: Complejidad temporal $O(logV)$.
- **Memoria adicional**: $O(V)$, sin sobrecosto de punteros.

10.3. Dijkstra sobre Grafo Dirigido con Bloqueo de Tráfico (`rutas.py`)

- **Complejidad Temporal**:
$$O((|V| + |E|)log|V|)$$
Donde $|V|$ es el número de celdas transitables de pasillo y $|E|$ es el número de transiciones permitidas según el sentido de circulación.

- **Caché Determinista**:
Dado que el grafo de pasillos es estático cuando no hay bloqueos de tráfico, las consultas de caminos mínimos entre celdas repetidas se cachean en memoria (`grafo._cache_dijkstra`). El tiempo de respuesta pasa de $\approx 1.8ms$ a $O(1)$ **instantáneo**, acelerando el cálculo inicial de la simulación.
- **Cálculo con Nodos Bloqueados**:
Cuando un picker detecta congestión, se invoca a Dijkstra pasando un conjunto de celdas prohibidas

`nodos_bloqueados`. En este escenario, la caché se elude dinámicamente y el algoritmo explora caminos alternativos en tiempo real.

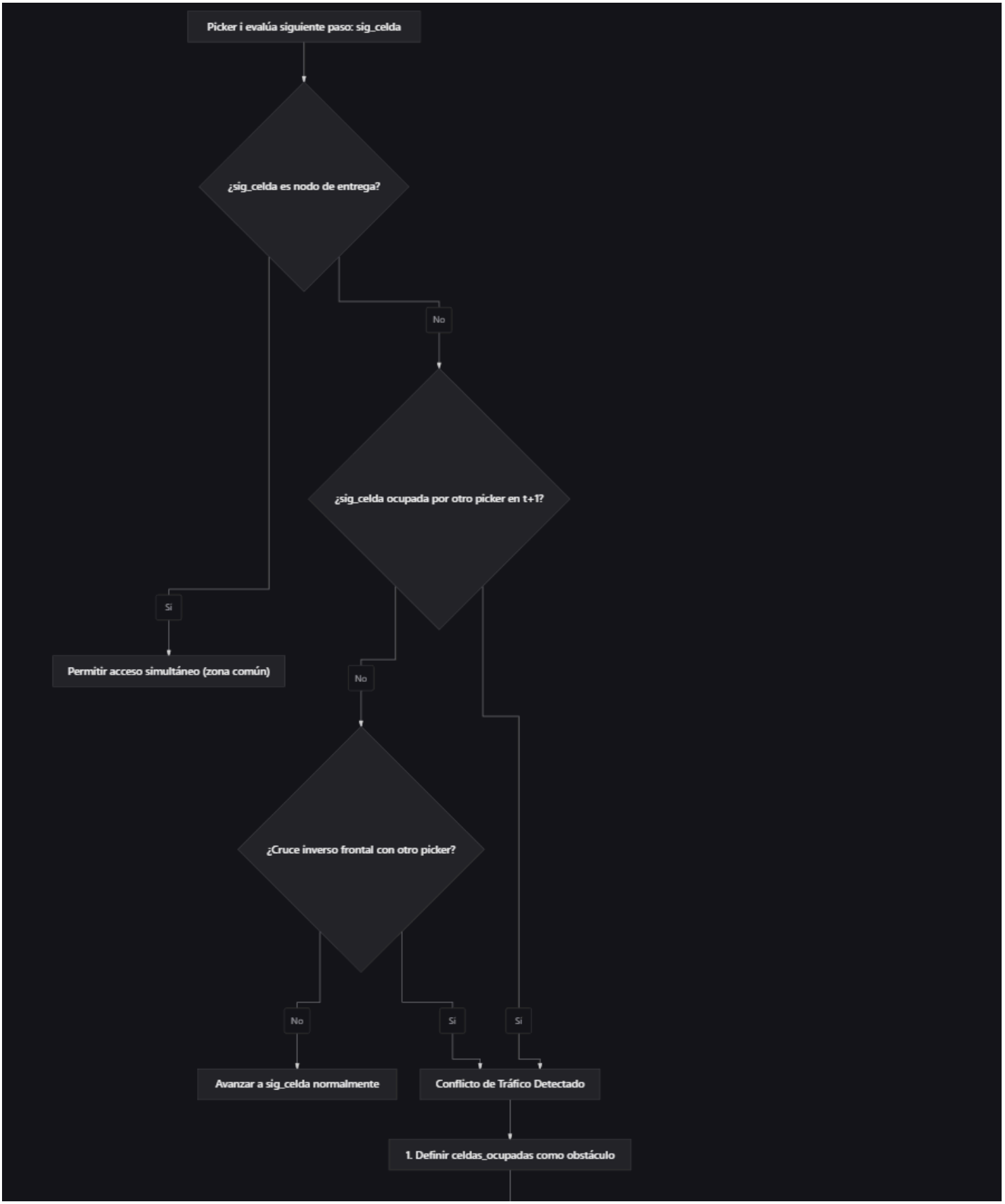
10.4. Ordenamiento Divide y Vencerás (`merge_sort`)

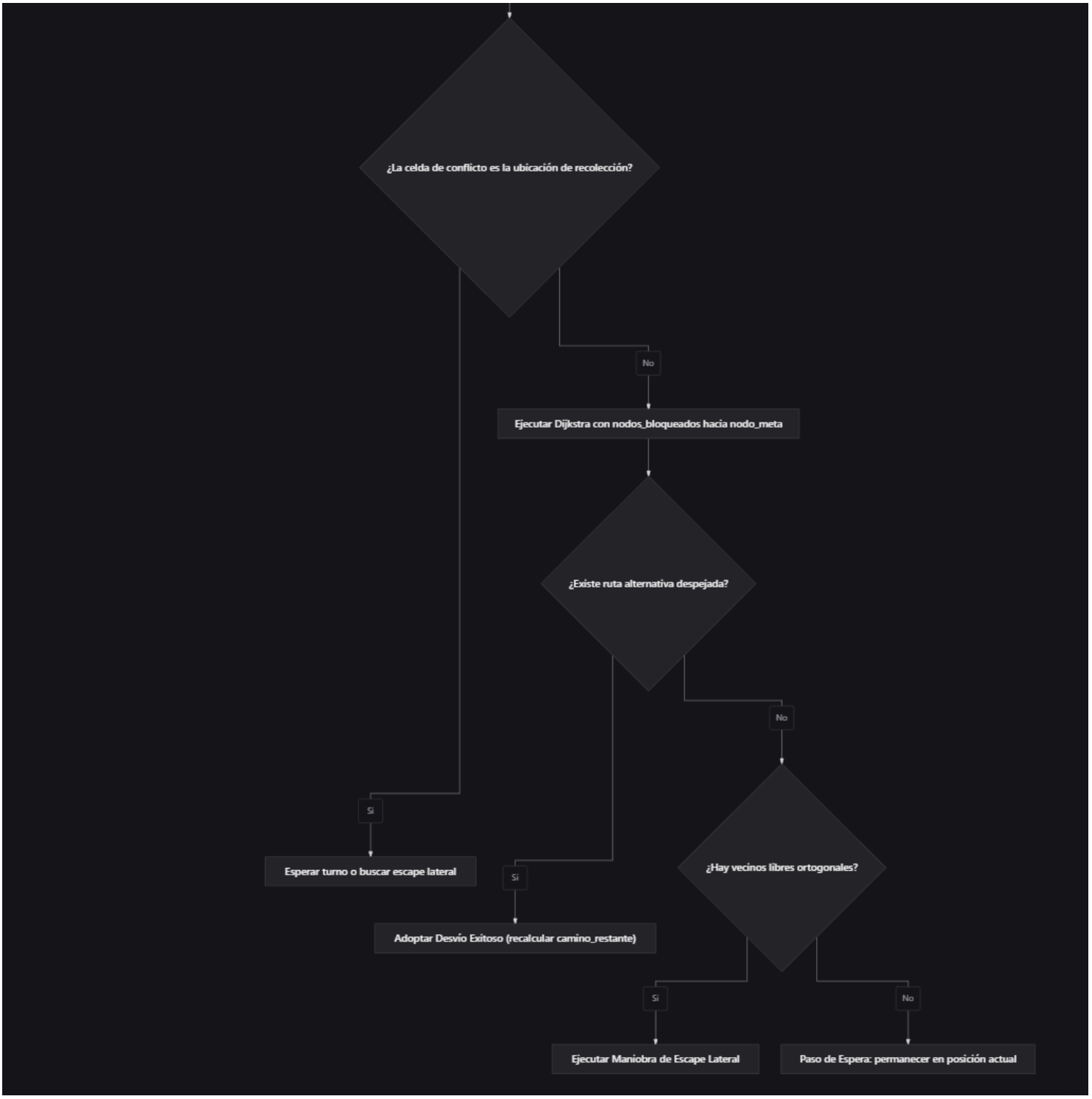
- **Estabilidad Algorítmica**:
  A diferencia de algoritmos inestables como QuickSort, `merge_sort` preserva el orden relativo de pedidos con idéntica prioridad.
- **Complejidad Temporal**: $O(NlogN)$ tanto en el mejor, promedio como peor caso.
- **Complejidad Espacial**: $O(N)$ debido a la combinación de sublistas (`_merge`).

---

11. Mecanismo de Evasión Dinámica de Tráfico y Protocolo Anticolisión

El corazón del módulo `coordinador.py` es su bucle de eventos discretos. En cada paso temporal $t \rightarrow t + 1$, el motor resuelve potenciales conflictos espaciales entre los pickers activos:


Picker i evalúa siguiente paso: sig_celda
¿sig_celda es nodo de entrega?
No
¿sig_celda ocupada por otro picker en t+1?
Si
Permitir acceso simultáneo (zona común)
No
¿Cruce inverso frontal con otro picker?
No
Si
Si
Avanzar a sig_celda normalmente
Conflicto de Tráfico Detectado
1. Definir celdas_ocupadas como obstáculo



Clasificación de Conflictos Resueltos:

1. **Conflicto de Concurrencia Directa (Mismo Destino)**:
Dos pickers intentan moverse a la misma celda (x, y) en el mismo instante. El picker con mayor prioridad de avance se adueña de la celda; el segundo activa el protocolo de evasión.

2. **Cruce Frontal Inverso (Head-on Collision)**:
El Picker A está en la celda C_1 y desea ir a C_2, mientras que el Picker B está en C_2 y desea ir a C_1. Si ambos avanzaran, chocarían a mitad de camino. El algoritmo detecta el intercambio cruzado y obliga a uno de ellos a desviar o maniobrar lateralmente.

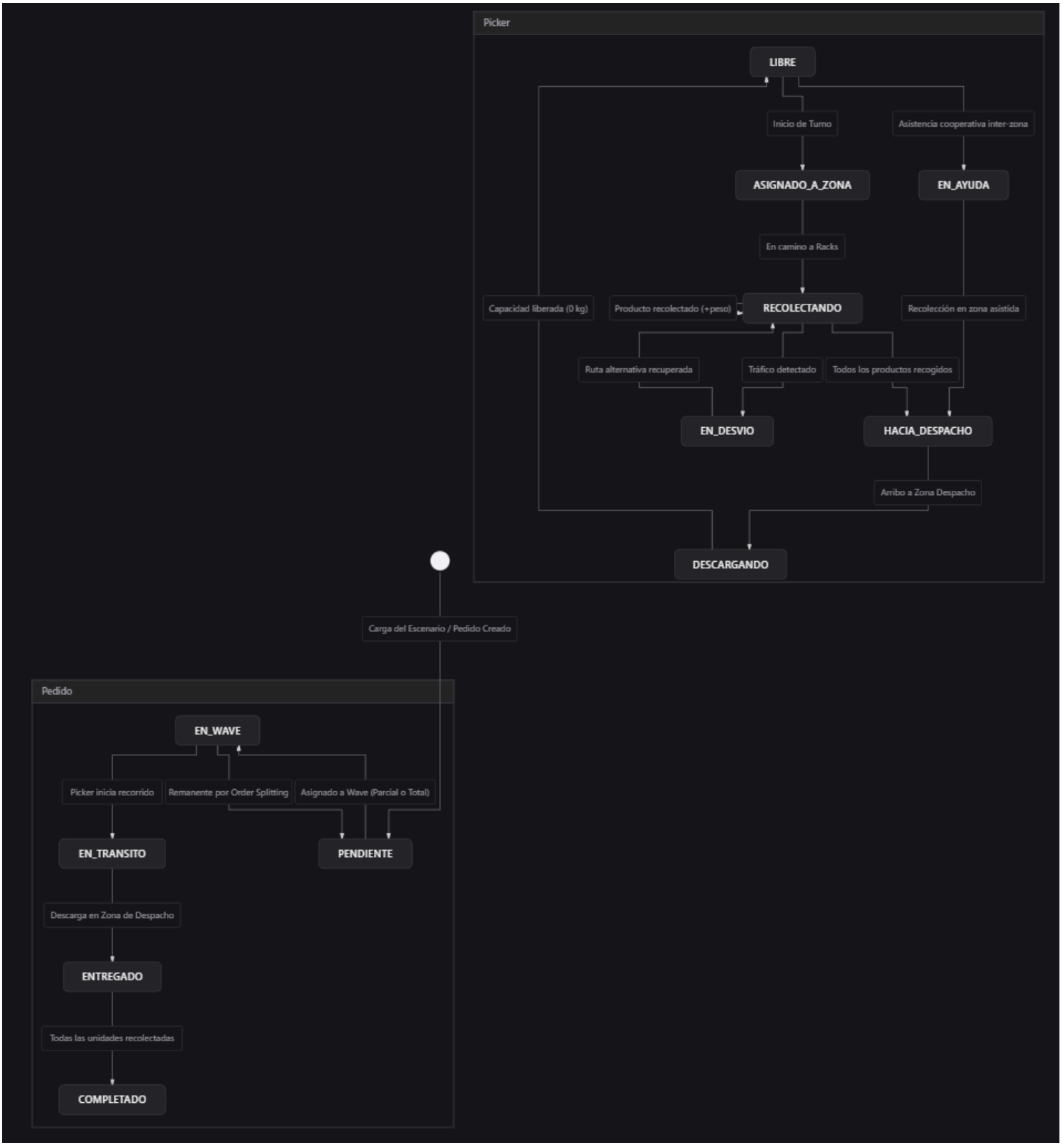
3. **Protección de Paradas de Recolección**:
Si la celda en disputa es el lugar donde el picker debe recolectar un producto (`primer_pick_idx == 1`), el

sistema prohíbe saltar dicha celda para evitar omitir ítems del pedido. En ese caso, el picker espera pacientemente a que el pasillo se desocupe o realiza una maniobra de aproximación en bucle.

---

12. Máquina de Estados del Negocio

El ciclo de vida de los pedidos, las olas y las tareas atómicas de recolección sigue una máquina de estados finitos que garantiza la trazabilidad operativa:



---

13. Comparativa Teórica y Práctica de las Estrategias de Ruteo

El sistema expone el método estático `CoordinadorMultiPicker.comparar_escenario_completo(...)`, el cual somete exactamente el mismo lote de pedidos a las tres estrategias bajo condiciones idénticas de tráfico y capacidad:

Dimensión de Análisis	Estrategia A: Orden de Pedidos	Estrategia B: Cercanía Greedy 3D	Estrategia C: Barrido por Pasillos (S-Shape)
Criterio de Ordenamiento	Cronológico según el archivo/cliente.	Minimización de distancia instantánea (Grafo + Z).	Geometría y sentido de circulación de las calles.
Comportamiento Espacial	Puede cruzar de un extremo a otro del almacén erráticamente.	Recorre en clústeres locales densos; puede dejar ítems aislados al final.	Patrón en zig-zag unidireccional y barrido bilateral continuo.
Tratamiento del Costo Z	No considera la altura al planificar el orden.	Penaliza productos en niveles superiores ($Z > 1$) en la heurística.	Visita niveles superiores agrupados dentro de la misma columna de estante.
Distancia Total Típica	**Alta** (100% como base de comparación).	**Baja** (reducción del 25% al 40% vs Estrategia A).	**Óptima / Muy Baja** (reducción del 30% al 45% y menor dispersión).
Riesgo de Congestión	Medio-Alto (trayectorias impredecibles).	Medio (los pickers pueden converger al mismo clúster denso).	**Mínimo** (el flujo ordenado por pasillos canaliza el tráfico en la misma dirección).
Costo Computacional	$O(K \cdot Dijkstra)$	$O(K^2 \cdot Dijkstra)$	$O(KlogK + K \cdot Dijkstra)$

---

14. Capa de Persistencia y Estructura del Esquema JSON

El módulo `persistencia.py` implementa el patrón Adaptador para convertir la representación en memoria del almacén a un esquema JSON serializable y viceversa.

Estructura del Documento JSON (`escenario_basico.json`):

```
{
  "warehouse": {
    "dimensiones": { "ancho_x": 18, "largo_y": 16 },
    "puertas": [
      { "id": "P1", "tipo": "OUTBOUND", "ubicacion": { "x": 3, "y": 0 } }
    ],
    "zonas_despacho": [
      { "id": "ZONA_A", "puerta_id": "P1", "ubicacion": { "x": 2, "y": 1 }, "ancho": 3, "alto": 2,
"peso_max_kg": 500.0 }
    ],
    "racks": [
      { "id": "R1", "ubicacion": { "x": 2, "y": 5 }, "ancho": 2, "alto": 6 }
    ],
    "pasillos": [
      { "id": "PAS_V1", "ubicacion": { "x": 0, "y": 5 }, "ancho": 2, "alto": 6, "sentido": "N" }
    ]
  },
  "pickers": [
    { "id": "PK1", "posicion_inicial": { "x": 4, "y": 3 }, "capacidad_carga_kg": 150.0 }
  ],
  "productos": [
    { "id": "PR01", "nombre": "Notebook 14", "ubicacion": { "x": 2, "y": 6, "z": 2 }, "cantidad_disponible":
25, "peso_kg": 2.2, "rack_id": "R1" }
  ],
  "pedidos": [
    {
      "id": "PED01",
      "prioridad": 10,
      "productos": [{ "id": "PR01", "cantidad": 2 }]
    }
  ]
}
```


Soporte para Guardado y Reanudación de Simulaciones

Cuando el usuario guarda una simulación en curso, `PersistenciaJSON.guardar_escenario(...)` inyecta la clave `estado_simulacion`:

- `paso_actual`: Número de paso exacto al pausar.
- `estrategia`: Nombre de la heurística activa.
- `cap_zonas_actual`: Inventario de peso remanente en cada zona de despacho.
- `planes_pickers`: Historial de posiciones, turnos y productos recolectados hasta ese instante.
  Al recargar el archivo, `AppVisual._restaurar_estado_simulacion(...)` reconstruye el escenario en el mismo fotograma sin recalcular la simulación desde cero.

---

15. Guía de Extensibilidad para Desarrolladores

La arquitectura modular permite expandir las capacidades del sistema sin modificar el núcleo de clases existente:

¿Cómo agregar una nueva Estrategia de Pathfinding?

1. Abra `wave_picking_src/rutas.py`.
2. Dentro de la clase `Pathfinding`, defina un nuevo método estático, por ejemplo:

```python
@staticmethod
def estrategia_d_algoritmo_genetico(warehouse, wave, peso_max_picker=None, pos_inicio=None):
    ubicaciones, z_por_coord = Pathfinding._extraer_objetivos_wave(warehouse, wave)
    # 1. Aplicar lógica de optimización propia sobre 'ubicaciones'
    orden_optimizado = mi_heuristica(ubicaciones)
    # 2. Utilizar el motor común de recorrido y despacho
    return Pathfinding._recorrer_lista_destinos(
        warehouse, orden_optimizado, z_por_coord,
        "D - Algoritmo Genético", getattr(wave, 'peso_total', 0.0), pos_inicio
    )
```

3. Registre la nueva estrategia en `CoordinadorMultiPicker.comparar_escenario_completo` y en el selector de la interfaz gráfica en `simulacion.py`.

¿Cómo integrar un nuevo medio de transporte (Vehículos AGV / Drones)?

1. En `modelos.py`, herede de `Picker` o agregue atributos como `velocidad_celdas_por_segundo` o `nivel_bateria`.
2. En `warehouse.py`, el grafo puede incorporar pesos diferenciados por tipo de vehículo en `agregar_arista(u, v, peso)`.
3. El `CoordinadorMultiPicker` consumirá directamente estos nuevos pesos mediante la misma interfaz de Dijkstra sin requerir cambios estructurales.